

Go Programming Lab

Program 1

Data Types and Variables

```
package main

import "fmt"

func main() {

    // Variable declarations

    var name string

    var age int

    var height float64

    var isStudent bool


    // Taking input from the user

    fmt.Print("Enter your name: ")

    fmt.Scan(&name)


    fmt.Print("Enter your age: ")

    fmt.Scan(&age)


    fmt.Print("Enter your height (in meters): ")

    fmt.Scan(&height)


    fmt.Print("Are you a student? (true/false): ")

    fmt.Scan(&isStudent)


    // Displaying output

    fmt.Println("\n--- User Information ---")

    fmt.Println("Name    :", name)

    fmt.Println("Age     :", age)

    fmt.Println("Height  :", height, "meters")
}
```

```

        fmt.Println("Student :", isStudent)

    // Displaying data types
    fmt.Println("\n--- Data Types ---")
    fmt.Printf("Type of name   : %T\n", name)
    fmt.Printf("Type of age    : %T\n", age)
    fmt.Printf("Type of height  : %T\n", height)
    fmt.Printf("Type of isStudent : %T\n", isStudent)
}

```

OUTPUT

```

PS C:\Users\1msccsc03> d:
PS D:\> go run s1.go
Enter your name: angel
Enter your age: 11
Enter your height (in meters): 22
Are you a student? (true/false): true

```

--- User Information ---

```

Name   : angel
Age    : 11
Height : 22 meters
Student : true

```

--- Data Types ---

```

Type of name   : string
Type of age    : int
Type of height  : float64
Type of isStudent : bool

```

Program 2

```
package main

import "fmt"

func main() {

    a := 10

    b := 5


    // Arithmetic Operators

    fmt.Println("Arithmetic Operations")

    fmt.Println("a + b =", a+b)

    fmt.Println("a - b =", a-b)

    fmt.Println("a * b =", a*b)

    fmt.Println("a / b =", a/b)

    fmt.Println("a % b =", a%b)


    // Relational Operators

    fmt.Println("\nRelational Operations")

    fmt.Println("a == b :", a == b)

    fmt.Println("a != b :", a != b)

    fmt.Println("a > b :", a > b)

    fmt.Println("a < b :", a < b)

    fmt.Println("a >= b :", a >= b)

    fmt.Println("a <= b :", a <= b)


    // Logical Operators

    x := true

    y := false


    fmt.Println("\nLogical Operations")

    fmt.Println("x && y :", x && y)

    fmt.Println("x || y :", x || y)
```

```
        fmt.Println("!x :", !x)
        fmt.Println("!y :", !y)
    }
```

Output:

PS S:\> go run s2.go

Arithmetic Operations

$a + b = 15$

$a - b = 5$

$a * b = 50$

$a / b = 2$

$a \% b = 0$

Relational Operations

$a == b$: false

$a != b$: true

$a > b$: true

$a < b$: false

$a >= b$: true

$a <= b$: false

Logical Operations

$x \&\& y$: false

$x || y$: true

$!x$: false

$!y$: true

Program 3

```
package main

import "fmt"

func main() {
    var n int

    fmt.Print("Enter the number of terms: ")

    fmt.Scan(&n)

    a, b := 0, 1

    if n <= 0 {
        fmt.Println("Please enter a positive number.")
    } else {
        fmt.Println("Fibonacci Series:")
        for i := 1; i <= n; i++ {
            fmt.Print(a, " ")

            next := a + b
            a = b
            b = next
        }
    }
}
```

OUTPUT

Enter the number of terms: 10

Fibonacci Series:

0 1 1 2 3 5 8 13 21 34

Program 4

```
package main
```

```
import "fmt"
```

```
// Recursive function to calculate factorial
```

```
func factorial(n int) int {  
    if n == 0 || n == 1 {  
        return 1  
    }  
    return n * factorial(n-1)  
}
```

```
func main() {  
    var num int  
  
    fmt.Print("Enter a number: ")  
    fmt.Scan(&num)  
    if num < 0 {  
        fmt.Println("Factorial is not defined for negative numbers.")  
    } else {  
        fmt.Printf("Factorial of %d is %d\n", num, factorial(num))  
    }  
}
```

OUTPUT

```
Enter a number: 5
```

```
Factorial of 5 is 120
```

Program 5

```
package main

import "fmt"

// Structure
type Student struct {
    Name string
    Roll int
    Marks [3]int // Array
}

func main() {

    // Structure with Array
    student := Student{
        Name: "John",
        Roll: 101,
        Marks: [3]int{85, 90, 95},
    }

    // Display Structure
    fmt.Println("Student Details")
    fmt.Println("Name :", student.Name)
    fmt.Println("Roll :", student.Roll)

    // Array
    fmt.Println("\nMarks (Array):")
    for i := 0; i < len(student.Marks); i++ {
        fmt.Printf("Subject %d: %d\n", i+1, student.Marks[i])
    }

    // Slice
    subjects := []string{"Math", "Science", "English"}
```

```
subjects = append(subjects, "Computer")

fmt.Println("\nSubjects (Slice):", subjects)


// Map
grades := map[string]string{
    "Math":  "A",
    "Science": "A",
    "English": "B+",
    "Computer": "A+",
}

fmt.Println("\nGrades (Map):")
for subject, grade := range grades {
    fmt.Println(subject, ":", grade)
}
}
```

Output

Student Details

Name : John

Roll : 101

Marks (Array):

Subject 1: 85

Subject 2: 90

Subject 3: 95

Subjects (Slice): [Math Science English Computer]

Grades (Map):

Math : A

Science : A

English : B+

Computer : A+

Program 6

```
package main
```

```
import "fmt"
```

```
func main() {
```

```
    arr := []int{45, 12, 78, 34, 23}
```

```
    key := 34
```

```
    // Bubble Sort
```

```
    for i := 0; i < len(arr)-1; i++ {
```

```
        for j := 0; j < len(arr)-i-1; j++ {
```

```
            if arr[j] > arr[j+1] {
```

```
                arr[j], arr[j+1] = arr[j+1], arr[j]
```

```
            }
```

```
        }
```

```
    }
```

```
    // Linear Search
```

```
    linear := -1
```

```
    for i := 0; i < len(arr); i++ {
```

```
        if arr[i] == key {
```

```
            linear = i
```

```
            break
```

```
        }
```

```
    }
```

```
    // Binary Search
```

```
    low, high, binary := 0, len(arr)-1, -1
```

```

for low <= high {
    mid := (low + high) / 2
    if arr[mid] == key {
        binary = mid
        break
    } else if arr[mid] < key {
        low = mid + 1
    } else {
        high = mid - 1
    }
}

fmt.Println("Sorted Array:", arr)
fmt.Println("Linear Search Index:", linear)
fmt.Println("Binary Search Index:", binary)

fmt.Println("\nComplexity:")
fmt.Println("Linear Search : O(n)")
fmt.Println("Binary Search : O(log n)")
fmt.Println("Bubble Sort : O(n²)")
fmt.Println("\nBest: Binary Search (O(log n))")
}

```

OUTPUT

Sorted Array: [12 23 34 45 78]

Linear Search Index: 2

Binary Search Index: 2

Complexity:

Linear Search : O(n)

Binary Search : O(log n)

Bubble Sort : $O(n^2)$

Best: Binary Search ($O(\log n)$)

program 7

```
package main
```

```
import "fmt"
```

```
// Recursive Function
```

```
func recursiveFib(n int) int {
```

```
    if n <= 1 {
```

```
        return n
```

```
    }
```

```
    return recursiveFib(n-1) + recursiveFib(n-2)
```

```
}
```

```
// Non-Recursive Function
```

```
func iterativeFib(n int) int {
```

```
    if n <= 1 {
```

```
        return n
```

```
    }
```

```
    a, b := 0, 1
```

```
    for i := 2; i <= n; i++ {
```

```
        c := a + b
```

```
        a = b
```

```
        b = c
```

```
    }
```

```
    return b
```

```
}
```

```

func main() {
    var n int

    fmt.Print("Enter the value of n: ")
    fmt.Scan(&n)

    fmt.Println("Recursive Fibonacci =", recursiveFib(n))
    fmt.Println("Non-Recursive Fibonacci =", iterativeFib(n))

    fmt.Println("\nComparison:")
    fmt.Println("Recursive Method")
    fmt.Println("Time Complexity : O(2^n)")
    fmt.Println("Space Complexity: O(n)")

    fmt.Println("\nNon-Recursive Method")
    fmt.Println("Time Complexity : O(n)")
    fmt.Println("Space Complexity: O(1)")

    fmt.Println("\nBetter Method: Non-Recursive")
    fmt.Println("Reason: It is faster and uses less memory.")
}

```

OUTPUT

Enter the value of n: 10

Recursive Fibonacci = 55

Non-Recursive Fibonacci = 55

Comparison:

Recursive Method

Time Complexity : O(2^n)

Space Complexity: $O(n)$

Non-Recursive Method

Time Complexity : $O(n)$

Space Complexity: $O(1)$

Better Method: Non-Recursive

Reason: It is faster and uses less memory.

Program 8

```
package main

import (
    "fmt"
    "time"
)

// Merge Sort
func mergeSort(arr []int) []int {
    if len(arr) <= 1 {
        return arr
    }

    mid := len(arr) / 2
    left := mergeSort(arr[:mid])
    right := mergeSort(arr[mid:])

    return merge(left, right)
}

// Merge Function
func merge(left, right []int) []int {
    result := []int{}
```

```

    for len(left) > 0 && len(right) > 0 {
        if left[0] < right[0] {
            result = append(result, left[0])
            left = left[1:]
        } else {
            result = append(result, right[0])
            right = right[1:]
        }
    }
    result = append(result, left...)
    result = append(result, right...)

    return result
}

// Quick Sort
func quickSort(arr []int) {
    if len(arr) < 2 {
        return
    }

    pivot := arr[len(arr)-1]
    left := 0

    for i := 0; i < len(arr)-1; i++ {
        if arr[i] < pivot {
            arr[i], arr[left] = arr[left], arr[i]
            left++
        }
    }

    arr[left], arr[len(arr)-1] = arr[len(arr)-1], arr[left]

```

```
        quickSort(arr[:left])
        quickSort(arr[left+1:])
    }
```

```
func main() {

    arr1 := []int{45, 12, 89, 23, 10}
    arr2 := []int{45, 12, 89, 23, 10}
    var mergeResult []int
    // Measure Merge Sort Time
    start := time.Now()
    for i := 0; i < 10000; i++ {
        mergeResult = mergeSort(arr1)
    }
    mergeTime := time.Since(start)
    // Measure Quick Sort Time
    start = time.Now()
    for i := 0; i < 10000; i++ {
        temp := make([]int, len(arr2))
        copy(temp, arr2)
        quickSort(temp)

        if i == 9999 {
            arr2 = temp
        }
    }
    quickTime := time.Since(start)
    // Display Results
    fmt.Println("Merge Sort Result :", mergeResult)
    fmt.Println("Merge Sort Time  :", mergeTime)
```

```

    fmt.Println()

    fmt.Println("Quick Sort Result :", arr2)

    fmt.Println("Quick Sort Time  :", quickTime)


    fmt.Println()


    if mergeTime < quickTime {
        fmt.Println("Merge Sort is Faster")
    } else if quickTime < mergeTime {
        fmt.Println("Quick Sort is Faster")
    } else {
        fmt.Println("Both algorithms took the same time")
    }
}

```

OUTPUT

Merge Sort Result : [10 12 23 45 89]

Merge Sort Time : 1.5182ms

Quick Sort Result : [10 12 23 45 89]

Quick Sort Time : 0s

Quick Sort is Faster

Program 9

```

package main

import "fmt"

// Define the structure of a Binary Tree Node
type Node struct {
    data int
    left *Node
    right *Node
}

// Preorder Traversal (Root -> Left -> Right)

```

```

func preorder(root *Node) {
    if root != nil {
        fmt.Print(root.data, " ")
        preorder(root.left)
        preorder(root.right)
    }
}

// Inorder Traversal (Left -> Root -> Right)
func inorder(root *Node) {
    if root != nil {
        inorder(root.left)
        fmt.Print(root.data, " ")
        inorder(root.right)
    }
}

// Postorder Traversal (Left -> Right -> Root)
func postorder(root *Node) {
    if root != nil {
        postorder(root.left)
        postorder(root.right)
        fmt.Print(root.data, " ")
    }
}

func main() {
    root := &Node{data: 10}
    root.left = &Node{data: 20}
    root.right = &Node{data: 30}
    root.left.left = &Node{data: 40}
    root.left.right = &Node{data: 50}
    root.right.left = &Node{data: 60}
    root.right.right = &Node{data: 70}
}

```

```

    fmt.Println("Binary Tree Traversals")

    fmt.Println("-----")

    fmt.Print("Preorder Traversal: ")
    preorder(root)

    fmt.Print("\nInorder Traversal: ")
    inorder(root)

    fmt.Print("\nPostorder Traversal: ")
    postorder(root)

    fmt.Println()
}

```

OUTPUT

Binary Tree Traversals

Preorder Traversal: 10 20 40 50 30 60 70

Inorder Traversal: 40 20 50 10 60 30 70

Postorder Traversal: 40 50 20 60 70 30 10

Program 10

```
package main
```

```
import "fmt"
```

```
func main() {
```

```
    // Input matrices
```

```
    A := [2][2]int{{1, 2}, {3, 4}}
```

```
    B := [2][2]int{{5, 6}, {7, 8}}
```

```
    // Elements of Matrix A
```

```

a, b := A[0][0], A[0][1]
c, d := A[1][0], A[1][1]

// Elements of Matrix B
e, f := B[0][0], B[0][1]
g, h := B[1][0], B[1][1]

// Seven products
p1 := a * (f - h)
p2 := (a + b) * h
p3 := (c + d) * e
p4 := d * (g - e)
p5 := (a + d) * (e + h)
p6 := (b - d) * (g + h)
p7 := (a - c) * (e + f)

// Result matrix
C := [2][2]int{}

C[0][0] = p5 + p4 - p2 + p6
C[0][1] = p1 + p2
C[1][0] = p3 + p4
C[1][1] = p1 + p5 - p3 - p7

// Display result
fmt.Println("Product Matrix:")
for i := 0; i < 2; i++ {
    for j := 0; j < 2; j++ {
        fmt.Print(C[i][j], "\t")
    }
    fmt.Println()
}

```

```
    }  
}
```

OUTPUT

Product Matrix:

```
19  22  
43  50
```

Program 11

```
package main  
  
import "fmt"  
  
func max(a, b int) int {  
    if a > b {  
        return a  
    }  
    return b  
}  
  
func main() {  
    // Weights and values of items  
    weight := []int{1, 3, 4, 5}  
    value := []int{1, 4, 5, 7}  
  
    capacity := 7  
    n := len(weight)  
    // DP table  
    dp := make([][]int, n+1)  
    for i := 0; i <= n; i++ {  
        dp[i] = make([]int, capacity+1)  
    }  
}
```

```

// Build DP table
for i := 1; i <= n; i++ {
    for w := 1; w <= capacity; w++ {
        if weight[i-1] <= w {
            dp[i][w] = max(value[i-1]+dp[i-1][w-weight[i-1]], dp[i-1][w])
        } else {
            dp[i][w] = dp[i-1][w]
        }
    }
}

fmt.Println("Maximum Profit =", dp[n][capacity])
}

```

OUTPUT

Maximum Profit = 9

Program 12

```

package main
import "fmt"
const INF = 99999
func printMatrix(title string, m [][]int) {
    fmt.Println(title)
    for _, row := range m {
        fmt.Println(row)
    }
}

func warshall(g [][]int) {
    n := len(g)
    reach := make([][]int, n)
    for i := range g {

```

```

        reach[i] = append([],int(nil), g[i]...)
    }

    for k := 0; k < n; k++ {
        for i := 0; i < n; i++ {
            for j := 0; j < n; j++ {
                if reach[i][k] == 1 && reach[k][j] == 1 {
                    reach[i][j] = 1
                }
            }
        }
    }

    printMatrix("Warshall's Algorithm (Transitive Closure):", reach)
}

func floydWarshall(g [][int])
{
    n := len(g)
    dist := make([][]int, n)
    for i := range g {
        dist[i] = append([]int(nil), g[i]...)
    }

    for k := 0; k < n; k++ {
        for i := 0; i < n; i++ {
            for j := 0; j < n; j++ {
                if dist[i][k]+dist[k][j] < dist[i][j] {
                    dist[i][j] = dist[i][k] + dist[k][j]
                }
            }
        }
    }
}

```

```

printMatrix("\nFloyd-Warshall Algorithm (Shortest Paths):", dist)
}

func main() {
    warshall([][]int{
        {1, 1, 0, 0}, {0, 1, 1, 0}, {0, 0, 1, 1}, {0, 0, 0, 1},
    })

    floydWarshall([][]int{
        {0, 3, INF, 7}, {8, 0, 2, INF}, {5, INF, 0, 1}, {2, INF, INF, 0},
    })
}

```

OUTPUT:

Warshall's Algorithm (Transitive Closure):

```

[1 1 1 1]
[0 1 1 1]
[0 0 1 1]
[0 0 0 1]

```

Floyd-Warshall Algorithm (Shortest Paths):

```

[0 3 5 6]
[5 0 2 3]
[3 6 0 1]
[2 5 7 0]

```

Program 13

```

package main
import ("fmt"; "sort")
const l = 99
func main() {
    c := [][]int{{0,2,1,6}, {2,0,3,8}, {1,3,0,1}, {6,8,1,0}}
    v := map[int]bool{0: true}
    fmt.Println("Prim:")
}

```

```

for e := 0; e < 3; e++ {
    m, x, y := l, 0, 0
    for i, vi := range v {
        if vi { for j := range c { if !v[j] && c[i][j] < m { m, x, y = c[i][j], i, j } } }
    }

    v[y] = true; fmt.Println(x, "-", y, ":", m)
}

ed := []struct{u, v, w int}{{0,1,2}, {0,3,6}, {1,2,3}, {1,3,8}}
sort.Slice(ed, func(i, j int) bool { return ed[i].w < ed[j].w })

p, f := []int{0, 1, 2, 3}, func(int) int{return 0}
f = func(i int) int { if p[i] == i { return i }; p[i] = f(p[i]); return p[i] }
fmt.Println("\nKruskal:")
for _, e := range ed {
    if r1, r2 := f(e.u), f(e.v); r1 != r2 { p[r1] = r2; fmt.Println(e.u, "-", e.v, ":", e.w) }
}

d, dv := []int{0, l, l, l}, make([]bool, 4)
for i := 0; i < 3; i++ {
    u := -1
    for j, vj := range dv { if !vj && (u == -1 || d[j] < d[u]) { u = j } }
    dv[u] = true
    for k, w := range c[u] { if w != l && d[u]+w < d[k] { d[k] = d[u] + w } }
}

fmt.Println("\nDijkstra:", d)
}

```

OUTPUT:

Prim:

0 - 1 : 2

1 - 2 : 3

0 - 3 : 6

Kruskal:

0 - 1 : 2

1 - 2 : 3

0 - 3 : 6

Dijkstra: [0 2 5 6]

Program 14

```
package main
```

```
import (
```

```
    "fmt"
```

```
    "sort"
```

```
    "strings"
```

```
)
```

```
type N struct {
```

```
    c string
```

```
    f int
```

```
    l, r *N
```

```
}
```

```
func main() {
```

```
    t := "huffman coding"
```

```
    f := map[string]int{}
```

```
    for _, ch := range t {
```

```
        f[string(ch)]++
```

```
    }
```

```
    var ns []*N
```

```
    for c, freq := range f {
```

```
        ns = append(ns, &N{c: c, f: freq})
```

```
    }
```

```
    for len(ns) > 1 {
```

```
        sort.Slice(ns, func(i, j int) bool { return ns[i].f < ns[j].f })
```

```

        ns = append([]*N{{f: ns[0].f + ns[1].f, l: ns[0], r: ns[1]}}, ns[2:]...)
    }

    codes := map[string]string{}
    var gen func(*N, string)
    gen = func(n *N, s string) {
        if n == nil { return }
        if n.l == nil && n.r == nil {
            codes[n.c] = s
            return
        }

        gen(n.l, s+"0")
        gen(n.r, s+"1")
    }

    gen(ns[0], "")
    var sb strings.Builder
    for _, ch := range t {
        sb.WriteString(codes[string(ch)])
    }

    fmt.Println("Codes:", codes)
    fmt.Println("Compressed:", sb.String())
}

```

OUTPUT:

```

Codes: map[:11110 a:1001 c:1111 d:1011 f:011 g:1000 h:0100 i:1101 m:1010 n:00 o:1100
u:0101]
Compressed: 01000101011011101010010011101111110010111101001000

```

Program 15

```
package main

import (
    "fmt"
    "math"
)

var (
    a      = [][]int{{0, 10, 15, 20}, {10, 0, 35, 25}, {15, 35, 0, 30}, {20, 25, 30, 0}}
    n, mc, bp = 4, math.MaxInt32, []int{}
    b        = make([]int, 4)
)

func q(r int) {
    if r == n {
        fmt.Println("N-Queens:", b)
        return
    }

    for c := 0; c < n; c++ {
        s := true
        for i := 0; i < r; i++ {
            if b[i] == c || i-r == b[i]-c || r-i == b[i]-c {
                s = false
                break
            }
        }

        if s {
            b[r] = c
            q(r + 1)
        }
    }
}
```

```

func t(p []int, v []bool, c int) {
    if len(p) == n {
        if tot := c + a[p[n-1]][0]; a[p[n-1]][0] > 0 && tot < mc {
            mc, bp = tot, append(append([]int{}, p...), 0)
        }
        return
    }
    for x := 0; x < n; x++ {
        if !v[x] && a[p[len(p)-1]][x] > 0 {
            vc, pc := append([]bool{}, v...), append(append([]int{}, p...), x)
            vc[x] = true
            if c+a[p[len(p)-1]][x] < mc {
                t(pc, vc, c+a[p[len(p)-1]][x])
            }
        }
    }
}

func main() {
    q(0)
    v := make([]bool, n)
    v[0] = true
    t([]int{0}, v, 0)
    fmt.Printf("TSP Min Cost: %d | Path: %v\n", mc, bp)
}

```

OUTPUT:

N-Queens: [1 3 0 2]

N-Queens: [2 0 3 1]

TSP Min Cost: 80 | Path: [0 1 3 2 0]